

Curso Inicial Front-End

Clase 10: Práctica

Integradora de

Maquetación

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

¡Es hora de la verdad! En esta clase no aprenderemos conceptos nuevos, sino que pondremos a prueba todo lo visto en el módulo de Maquetación. Construiremos una **Landing Page Profesional** desde cero, aplicando semántica, **flexbox**, diseño responsivo y animaciones, consolidando tus habilidades como maquetador frontend.



Objetivos

- Integrar HTML5, CSS3, **Flexbox** y Responsive Design en un solo proyecto real.
- Aplicar un flujo de trabajo profesional: del diseño a la estructura y luego al estilo.
- Resolver problemas de maquetación complejos en un entorno de práctica supervisada.



Bloques Temáticos

- Tema 1: Desarrollo de una landing page completa desde cero integrando HTML5 y CSS3.
 - La Mentalidad del Desarrollador: No te abrumes
 - Puesta en Marcha: Tu Entorno de Trabajo
 - Guía de Construcción (Paso a Paso Sugerido)
 - El Truco Maestro del Borde Rojo
 - Lista de Verificación (Checklist de Éxito)
 - Arquitectura **CSS**: Ordena tu Cajón de Herramientas

¡Felicidades! Has llegado al final del primer gran bloque de tu carrera como desarrollador Frontend. Hasta ahora hemos visto muchas herramientas aisladas: aprendimos a poner los ladrillos (**HTML** Semántico), pintar las paredes (**CSS**), armar las habitaciones (**Flexbox**), hacer que la casa se achique mágicamente (Responsive) y darle vida con animaciones.

Hoy no aprenderemos ningún concepto teórico nuevo. Hoy nos pondremos el casco de obra: uniremos todo lo aprendido para construir una página web profesional desde cero, enfrentándonos al famoso "Síndrome de la hoja en blanco".

Tema 1: Desarrollo de una landing page completa desde cero integrando HTML5 y CSS3.

1. La Mentalidad del Desarrollador: No te abrumes

Hacer una página desde cero puede parecer una montaña imposible de escalar. Pero los profesionales no suben la montaña de un solo salto heroico; la suben paso a paso.

- **El principio de la Fealdad:** No intentes que la web se vea bonita en los primeros 10 minutos. Es un error de principiante intentar darle color a algo que no existe. Primero cocina el **HTML** (lo estructural y feo) y luego decora con **CSS** (lo lindo).
- **La Analogía de la Casa:** No puedes pintar las paredes (**CSS**) si primero no has levantado las vigas de metal, echado el cemento y puesto el techo (**HTML**).

2. Puesta en Marcha: Tu Entorno de Trabajo

Abre VS Code y prepara tu lienzo de trabajo profesional:

1. Crea una carpeta llamada **proyecto-integrador**.

2. Dentro, crea tu `index.html`.
3. Crea una carpeta `css/` y dentro un archivo `styles.css`.
4. Crea una carpeta `img/` para guardar tus fotos descargadas.
5. Abre `index.html`, escribe `!` y presiona `Tab` para generar el Boilerplate.
6. ¡No lo olvides! Vincula tu **CSS** usando `<link rel="stylesheet" href="css/styles.css">` dentro del `<head>`.

3. Guía de Construcción (Paso a Paso Sugerido)

Paso 1: El Esqueleto Semántico (HTML Puro)

Crea las secciones vacías con sus respectivos IDs y clases. Solo pon los textos de prueba ("Lorem ipsum").

```
<header class="main-header">
  <nav>...</nav>
</header>

<main class="page-content">
  <section id="hero">...</section>
  <section id="proyectos">...</section>
  <section id="contacto">...</section>
</main>

<footer class="main-footer">...</footer>
```

Paso 2: El Reset y las Variables (CSS)

Ve a tu archivo `styles.css`. Limpia los márgenes sucios que el navegador trae por defecto y define la paleta de colores corporativa que usarás.

```
/* El Reset Universal Obligatorio */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* Guardamos colores en "Variables" para no tener que memorizar códigos Hexadecimales */
:root {
  --color-primario: #2563eb;
  --color-oscuro: #1e293b;
  --color-claro: #f8fafc;
}

body {
  background-color: var(--color-claro);
  font-family: 'Arial', sans-serif;
}
```

Paso 3: Mentalidad Mobile First

Escribe el **CSS** pensando exclusivamente en cómo se vería en un celular estrecho. - Todo debe estar apilado verticalmente (`flex-direction: column`).

- Las imágenes grandes deben tener `max-width: 100%` para no romper la pantalla.

Comparativa Estratégica en Entrevistas: Mobile First vs Desktop First

Paradigma	¿Por dónde empiezo a diseñar?	¿Qué Media Query uso?	Veredicto Industrial
Mobile First	Celular (Pantalla pequeña e incómoda).	Uso <code>min-width</code> . Voy añadiendo columnas a medida que la pantalla tiene espacio para crecer.	✅ Estándar actual absoluto. Escribes menos código y la web carga más rápido en redes 3G lentas.
Desktop First	PC (Monitor gigante de 24 pulgadas).	Uso <code>max-width</code> . Voy "destruyendo" columnas y amontonando cosas para que entren en el celular.	❌ Obsoleto. Obligas al celular a procesar un diseño de escritorio complejo que luego tendrá que ocultar.

Paso 4: Añadiendo Lujos (Breakpoints y Media Queries)

Una vez que el celular se ve bien, usa `@media (min-width: 768px)` para la Tablet, y `@media (min-width: 1024px)` para PC. En esos bloques es donde cambias tus **Flexbox** a `row` para que los proyectos se acomoden uno al lado del otro.

4. El Truco Maestro del Borde Rojo

Todos pasamos por esto: Una caja no aparece donde quieres, **Flexbox** parece ignorarte, o hay un misterioso espacio en blanco que hace scroll horizontal. Antes de frustrarte, aplica este "Rayo X" temporal al inicio de tu **CSS**:

```
* {  
  border: 1px solid red !important;  
}
```

Esto dibujará una línea roja sobre **los límites reales de cada caja invisible**. Te darás cuenta instantáneamente si un `div` está desbordando o si te olvidaste de cerrarlo. Una vez que encuentres al culpable, borras esta regla.

Checkpoint de Comprensión: Estás construyendo la grilla de proyectos. Tienes 3 tarjetas. En celular se ven una debajo de la otra. Haces tu `@media (min-width: 768px) { .grilla { display: flex; flex-direction: row; } }`. Pero notas que las tarjetas se estiran hasta ocupar todo el ancho, rompiéndose horriblemente. ¿Qué propiedad mágica de `max-width` o `flex-basis` le faltó a las hijas para respetar su tamaño máximo?

5. Lista de Verificación (Checklist de Éxito)

Antes de entregar un proyecto a un cliente (o a tu profesor), tilda estos 5 mandamientos:

1. ☐ ¿Tengo un solo `<h1>`? (Afecta el SEO si tienes varios).
2. ☐ ¿Mis botones tienen el cursor de la manito? (Agrega `cursor: pointer` en **CSS**).
3. ☐ ¿Todas mis imágenes `<img...>` tienen un atributo `alt="descripción"` obligatorio?
4. ☐ ¿Está vivo mi Responsive? (Al achicar la ventana arrastrando con el mouse, nada debe generar una barra de scroll horizontal en la base).
5. ☐ ¿Mi código es válido? (Copia todo tu **HTML** y pégalo en validator.w3.org para descubrir si dejaste etiquetas sin cerrar que luego romperán **JavaScript**).

6. Arquitectura CSS: Ordena tu Cajón de Herramientas

En proyectos reales inmensos, no metemos 2000 líneas de código a lo loco. Dividimos el código en secciones lógicas usando grandes comentarios visuales en nuestro `styles.css`:

1. `/* === RESET Y VARIABLES === */` (Tus colores y fuentes universales).
 2. `/* === ESTRUCTURA GLOBAL (LAYOUT) === */` (Diseño del header, main, y footer).
 3. `/* === COMPONENTES === */` (Diseño de botones, tarjetas o inputs. Cosas que se repiten).
 4. `/* === UTILIDADES === */` (Clases rápidas como `.texto-centrado { text-align: center; }`).
-

Documentación Oficial (Referencias)

- CSS: <https://developer.mozilla.org/es/docs/Web/CSS>
- Flexbox: https://developer.mozilla.org/es/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox
- GitHub: <https://docs.github.com/es>
- HTML: <https://developer.mozilla.org/es/docs/Web/HTML>
- JavaScript: <https://developer.mozilla.org/es/docs/Web/JavaScript>